字段：字段名称。

`Field` 可以指定一个默认值（也可以是一个工厂函数）。如果不指定默认值，则值 `None` 会被分配给字段。如果字段名称以下划线开头，则它被视为私有属性，不会被模型系统处理。

示例代码

```
class User(BaseModel):
    id: int = Field(default_factory=lambda: 99, ...)
    _internal_id: int = 42
    temporary_value = "not a field"

print(User(id=1).model_dump())
# 输出: {'id': 1}
```

输出结果：表明模型只包含 `id`。

```
class Foo(BaseModel):
    a: int = 1
    b = 2

print(Foo.__pydantic_fields__.keys())
# 输出: dict_keys(['a'])
```

这段代码表明：

- `a` 是一个模型字段，因为它有类型注解。
- `b` 不是模型字段，因为它没有类型注解。

`__pydantic_fields__` 是一个内部属性，用于存储模型的字段信息。

`Field` 函数的参数可以用来定义字段的验证规则、默认值、别名、JSON Schema 信息等。下面是一些常用的参数：

- `default`：设置字段的默认值。
- `default_factory`：一个可调用对象，用于生成默认值。
- `alias`：为字段设置一个别名，用于序列化和反序列化。
- `description`：为字段添加描述，用于生成 JSON Schema。
- `gt`、`ge`、`lt`、`le`：数值大小限制（大于、大于等于、小于、小于等于）。
- `min_length`、`max_length`：字符串或列表的长度限制。
- `pattern`：字符串必须匹配的正则表达式。
- `exclude`：序列化时排除该字段。
- `repr`：是否在模型的 `__repr__` 输出中显示该字段。

示例代码

```
class User(BaseModel):
    id: int = Field(default=1, description="User ID")
    name: str = Field(alias="full_name", min_length=2)
    _internal_id: int = 42

print(User(id=1, full_name="Alice").model_dump())
# 输出: {'id': 1, 'name': 'Alice'}
```

这段代码定义了一个模型，其中 `id` 字段有默认值和描述，`name` 字段有别名和最小长度限制，`_internal_id` 是私有属性，不会被序列化。

`constrained` 系列函数的替代

`pydantic.v1` 中提供了一系列 `constrained` 函数（如 `constrint`、`constfloat`、`conlist`）来创建带约束的类型。在 `pydantic v2` 中，这些函数被更简洁、更强大的 `Annotated` 语法和 `Field` 函数所取代。

示例代码

```
# pydantic v1 风格
from pydantic.v1 import constrint, conlist
MyInt = constrint(gt=0, lt=100)
MyList = conlist(str, min_items=1, max_items=5)

class User(BaseModel):
    id: MyInt
    tags: MyList

# pydantic v2 风格
from typing_extensions import Annotated
from pydantic import Field
MyInt = Annotated[int, Field(gt=0, lt=100)]
MyList = Annotated[list[str], Field(min_items=1, max_items=5)]

class User(BaseModel):
    id: MyInt
    tags: MyList
```

`pydantic.v1` 的方式通过调用函数来生成类型，而 `pydantic v2` 的方式直接使用 `Annotated` 和 `Field` 来定义带约束的类型，更加简洁和直观。

`__pydantic_extra__` 属性

`__pydantic_extra__` 属性用于存储模型实例中，通过 `extra='allow'` 配置允许的、但未在模型中定义的额外字段。

示例代码

```
class Foo(BaseModel):
    a: int
    b: int

foo = Foo(a=1, b=2, c=3, d=4)
print(foo.__pydantic_extra__)
# 输出: {'c': 3, 'd': 4}
```

这段代码表明，`c` 和 `d` 是模型 `Foo` 的额外字段，它们被存储在 `__pydantic_extra__` 中。

`__pydantic_fields__` 属性

`__pydantic_fields__` 属性用于存储模型的字段信息。

示例代码

```
class Foo(BaseModel):
    a: int = 1
    b = 2

print(Foo.__pydantic_fields__.keys())
# 输出: dict_keys(['a'])
```

这段代码表明，`__pydantic_fields__` 只包含 `a`，因为 `b` 没有类型注解，不被视为模型字段。

`__pydantic_private__` 属性

`__pydantic_private__` 属性用于存储模型实例的私有属性。

示例代码

```
class Foo(BaseModel):
    _a: int = 42
    b: int = 1

foo = Foo(b=2)
print(foo.__pydantic_private__)
# 输出: {'_a': 42}
```

这段代码表明，`_a` 是模型 `Foo` 的私有属性，其值为 `42`。

`__pydantic_validator__` 属性

`__pydantic_validator__` 属性用于存储模型的验证器。

示例代码

```
class Foo(BaseModel):
    a: int = 1

print(Foo.__pydantic_validator__)
# 输出: <pydantic_core._pydantic_core.SchemaValidator object at 0x...>
```

这段代码表明，`__pydantic_validator__` 是一个 `SchemaValidator` 对象，用于执行模型的验证。

`__pydantic_serializer__` 属性

`__pydantic_serializer__` 属性用于存储模型的序列化器。

示例代码

```
class Foo(BaseModel):
    a: int = 1

print(Foo.__pydantic_serializer__)
# 输出: <pydantic_core._pydantic_core.SchemaSerializer object at 0x...>
```

这段代码表明，`__pydantic_serializer__` 是一个 `SchemaSerializer` 对象，用于执行模型的序列化。

`__pydantic_core_schema__` 属性

`__pydantic_core_schema__` 属性用于存储模型的核心验证模式。

示例代码

```
class Foo(BaseModel):
    a: int = 1

print(Foo.__pydantic_core_schema__)
# 输出: {'type': 'model', 'cls': <class '__main__.Foo'>, 'schema': {'type': 'model-fields', 'fields': {'a': {'type': 'field', 'schema': {'type': 'int'}}}}, ...}
```

这段代码表明，`__pydantic_core_schema__` 是一个内部数据结构，用于定义模型的验证规则。

本文由简悟 Chrome 扩展 Markdown 导出
来自：https://www.zhihu.com/question/19920...
时间：2026-03-01 09:22:52
用户：ID: 699250

`model_fields` 属性

`model_fields` 属性用于存储模型的字段信息。

示例代码

```
class Foo(BaseModel):
    a: int = 1

print(Foo.model_fields.keys())
# 输出: dict_keys(['a'])
```

这段代码表明，`model_fields` 只包含 `a`，因为 `b` 没有类型注解，不被视为模型字段。

`model_computed_fields` 属性

`model_computed_fields` 属性用于存储模型的计算字段。

示例代码

```
class Foo(BaseModel):
    a: int = 1

print(Foo.model_computed_fields)
# 输出: {}
```

这段代码表明，`model_computed_fields` 是一个空字典，因为模型 `Foo` 没有计算字段。

`model_extra` 属性

`model_extra` 属性用于存储模型实例中，通过 `extra='allow'` 配置允许的、但未在模型中定义的额外字段。

示例代码

```
class Foo(BaseModel):
    a: int
    b: int

foo = Foo(a=1, b=2, c=3, d=4)
print(foo.model_extra)
# 输出: {'c': 3, 'd': 4}
```

这段代码表明，`c` 和 `d` 是模型 `Foo` 的额外字段，它们被存储在 `model_extra` 中。

`model_fields_set` 属性

`model_fields_set` 属性用于存储在模型实例初始化时，显式设置的字段。

示例代码

```
class Foo(BaseModel):
    a: int = 1
    b: int = 2

foo = Foo(a=1, b=3)
print(foo.model_fields_set)
# 输出: {'a', 'b'}
```

这段代码表明，`a` 和 `b` 是在初始化时显式设置的字段。

`model_post_init` 钩子

`model_post_init` 钩子用于在模型实例初始化完成后，执行一些额外的操作。

示例代码

```
class Foo(BaseModel):
    a: int = 1

    def model_post_init(self, __context):
        print(f"Initialized with {self.a}")

foo = Foo(a=2)
# 输出: Initialized with 2
```

这段代码表明，`model_post_init` 钩子在模型实例初始化完成后，会执行一个打印操作。

`model_rebuild()` 方法

`model_rebuild()` 方法用于重新构建模型的验证器和序列化器。

示例代码

```
class Foo(BaseModel):
    a: int = 1

print(Foo.model_rebuild())
# 输出: True
```

这段代码表明，`model_rebuild()` 方法返回 `True`，表示模型成功重新构建。

`model_validate()` 方法

`model_validate()` 方法用于从一个字典或对象中，创建模型实例。

示例代码

```
class Foo(BaseModel):
    a: int = 1

print(Foo.model_validate({'a': 2}))
# 输出: a=2
```

这段代码表明，`model_validate()` 方法从字典 `{'a': 2}` 中创建了模型实例 `a=2`。

`model_validate_strings()` 方法

`model_validate_strings()` 方法用于从一个字符串中，创建模型实例。

示例代码

```
class Foo(BaseModel):
    a: int = 1

print(Foo.model_validate_strings('{"a": 2}'))
# 输出: a=2
```

这段代码表明，`model_validate_strings()` 方法从字符串 `'{"a": 2}'